TP02: How to Work with Big Data Files (+5GB)

Module: Big Data Analytics

Your Name
Second-year Master: IA & Data Science
University of Eloued

October 20, 2026

Abstract

This report studies three ways of processing a CSV file that is larger than the available memory: a standard `pandas.read_csv()`, chunked processing with pandas, and out-of-core parallel processing with Dask. We compare them on execution time and memory footprint, using a flight-itineraries dataset of 28.96 GB.

Contents

1 Introduction and Objectives **2**

2 Experimental Setup **2**
2.1 Dataset . . . 2
2.2 Environment . . . 2
2.3 Measurement methodology . . . 2

3 Task 1: The Traditional Failure (OOM) **2**
3.1 Method . . . 2
3.2 Result . . . 2

4 Task 2: Stream Processing (Pandas Chunking) **3**
4.1 Method . . . 3
4.2 Result . . . 3

5 Task 3: Parallelized Out-of-Core Compute (Dask) **3**
5.1 Method . . . 3
5.2 Lazy evaluation . . . 3
5.3 Result . . . 3

6 Comparison and Discussion **3**

7 Conclusion **4**

1 Introduction and Objectives

Traditional data-analysis tools load the whole dataset in RAM. When the file exceeds the memory of the machine, this approach fails. The objectives of this lab are:

- understand the Out-Of-Memory (OOM) limits of `pd.read_csv()`;
- implement chunking with pandas;
- use parallel and lazy computing with Dask DataFrames;
- benchmark execution time versus memory footprint.

2 Experimental Setup

2.1 Dataset

The dataset is `itineraries.csv` (flight itineraries), of size 28.96 GB. Only the columns `totalFare` (tasks 2 and 3) and `baseFare` (task 3) are needed.

2.2 Environment

Operating system	Linux-6.1.0-54-amd64-x86_64-with-glibc2.36
Logical CPU cores	4
Total RAM	11.6 GB
Python / pandas / Dask	3.11.2 / 3.0.6 / 2026.8.0

Table 1: Hardware and software environment.

2.3 Measurement methodology

Each script is run in its own Python process by a supervisor script (`benchmark_and_plot.py`). The supervisor samples the resident set size (RSS) of the process and its children every 0.1 s using `psutil`, and measures wall-clock time. Running each method in a fresh process guarantees that memory used by one method does not pollute the measurement of another.

3 Task 1: The Traditional Failure (OOM)

3.1 Method

We attempt to load the whole file with `pd.read_csv()`. To reproduce the failure safely, the address space of the process is limited to 4 GB with `resource.setrlimit(RLIMIT_AS)`, so that Python raises a `MemoryError` instead of freezing the machine. `missing: task1_oom.py`

3.2 Result

The process ended after 22.7 s with a peak RAM of 3710 MB. The outcome (crash or success) is given in Table 2. Pandas stores the data in memory as Python/NumPy objects, and string columns (`object` dtype) are especially expensive, so the in-memory size is typically much larger than the file on disk.

4 Task 2: Stream Processing (Pandas Chunking)

4.1 Method

The file is read in chunks of 100 000 rows, only the needed column is loaded (`usecols`), and the sum and row count are updated incrementally. The global mean is obtained at the end as $\bar{x} = \frac{\sum_i x_i}{N}$. Only one chunk is in memory at a time, so memory usage is bounded by the chunk size, not by the file size. `missing: task2_chunking.py`

4.2 Result

Chunking finished in 233.4 s with a peak RAM of 148 MB (average 138 MB). Memory stays roughly constant over time (Figure 3), but processing is sequential and uses a single core.

5 Task 3: Parallelized Out-of-Core Compute (Dask)

5.1 Method

`missing: task3_dask.py` The CSV is split into blocks of 128 MB, giving 242 partitions. Each partition is a regular pandas DataFrame that is processed independently by a pool of worker threads.

5.2 Lazy evaluation

Calling `dd.read_csv()`, `.mean()` or `.sum()` does not read any data: Dask only builds a *task graph* (a directed acyclic graph) describing the operations. Computation starts only when `compute()` is called. This has several benefits:

- the graph can be *optimized* before execution (e.g. column projection, fusion of tasks);
- independent tasks can be run in parallel;
- several results can be computed together with a single `dask.compute(a, b)`, so the file is read only once for both the mean and the sum.

missing: fig_dask_graph_raw.png

missing: fig_dask_graph_opt.png

Figure 1: Task graph (2 partitions): before (left) and after (right) optimization. Remove this figure if graphviz is not available.

5.3 Result

Dask finished in 208.3 s with a peak RAM of 1355 MB (average 1143 MB).

6 Comparison and Discussion

Method	Peak RAM (MB)	Avg RAM (MB)	Time (s)	Outcome
Standard Pandas	3710	1892	22.7	OOM / crash
Pandas Chunking	148	138	233.4	Success
Dask DataFrame	1355	1143	208.3	Success

Table 2: Measured performance of the three methods.

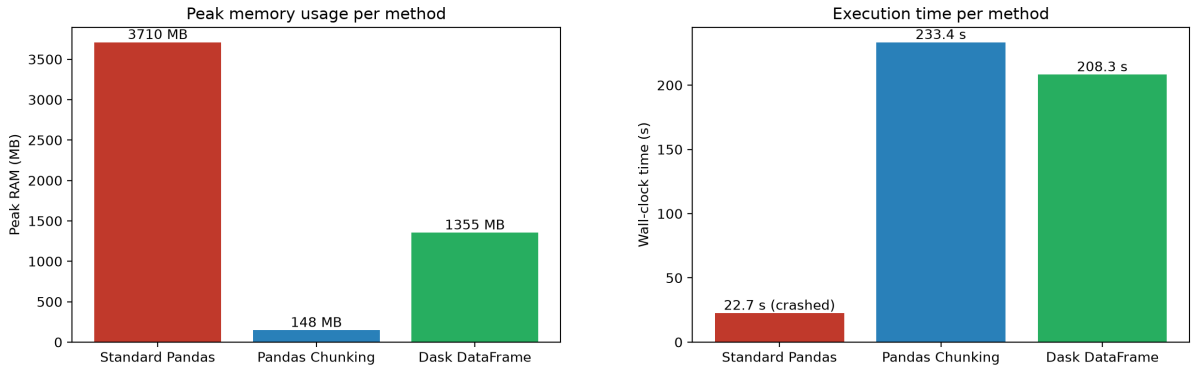


Figure 2: Peak RAM (left) and execution time (right).

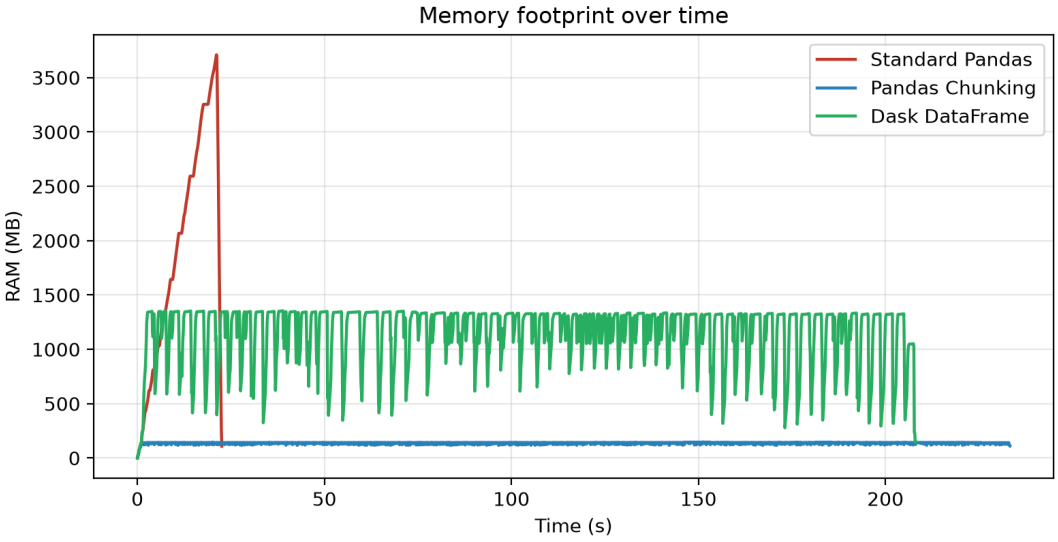


Figure 3: Memory footprint over time for the three methods.

Discussion. Standard pandas is bounded by the RAM (here limited to 4 GB) and cannot process a file larger than memory. Chunking removes this limitation at the price of sequential execution. Dask also keeps memory bounded, because partitions are streamed, but it additionally uses several cores. Its memory peak is generally higher than chunking because several partitions of 128 MB are in memory at the same time (one per worker thread), and it adds scheduling overhead; the speed-up is therefore most visible when the computation is CPU-bound or the disk is fast. Read speed from disk, the size of the selected columns, and the number of cores all influence the final ranking.

7 Conclusion

Loading a file larger than RAM with `pd.read_csv()` fails. Chunking solves the memory problem simply and with almost constant memory. Dask provides a higher-level, lazy and parallel framework that scales to bigger data and more complex computations with the same code style as pandas.

Reproducibility. All code is available on the GitHub repository of the student.